# U-AUBEN : guide pour l'équipe back-end

Plateforme des archives universitaires, partie front (Angular). Ce document explique comment lancer le front sur votre machine et quelles API il attend.

## 1. À quoi sert ce document

Le front est terminé pour une démonstration, mais ses données sont simulées dans le navigateur. Votre travail est de fournir l'API qui les remplace. Le front se branchera ensuite sur vos routes sans changer ses pages.

## 2. Lancer le front sur votre machine

### Prérequis

- **Node.js** avec une version récente. La CLI Angular du projet refuse de démarrer avec une version trop ancienne et affiche la version minimale exigée (au moment de l'écriture : Node 22.22.3 ou plus, ou 24.15.0 ou plus, ou 26.0.0 ou plus). Installez la version LTS la plus récente.
- **npm**, installé avec Node.
- **Git**, si le projet est fourni par dépôt.

Vérification :

```
node -v
npm -v
```

### Installation et démarrage

```
cd site-archives-u-auben
npm install
npx ng serve
```

Le site est alors disponible sur `http://localhost:4200`. Il se recharge tout seul à chaque modification de fichier. Pour arrêter le serveur : Ctrl + C.

Le dossier `node_modules` n'est pas dans le projet : `npm install` le recrée. Ne le copiez pas d'une machine à l'autre.

### Comptes de test (provisoires)

| Identifiant | Mot de passe | Rôle |
|---|---|---|
| etudiant | etudiant123 | Étudiant |
| chef.info | chef12345 | Chef de filière |
| admin | admin12345 | Administrateur |

Ces comptes sont écrits en dur dans `src/app/pages/connexion/connexion.ts`. **Ils doivent être supprimés** dès que la vraie connexion existe : leurs mots de passe sont visibles par tout le monde.

## 3. Organisation du code

| Fichier ou dossier | Rôle |
|---|---|
| `src/app/pages/` | Une page par dossier (connexion, tableau de bord, recherche, programme, etc.) |
| `src/app/services/donnees.ts` | **Point unique à brancher sur l'API** : modules, archives, programmes, comptes, téléchargements |
| `src/app/services/authentification.ts` | Session de l'utilisateur et son rôle |
| `src/app/gardes/gardes.ts` | Blocage des pages selon la connexion et le rôle |
| `src/app/app.routes.ts` | Liste des pages et de leurs adresses |

Le service `donnees.ts` contient aujourd'hui des listes d'exemple. Pour brancher l'API, on remplace le contenu de ses méthodes par des requêtes HTTP. Les pages n'ont presque rien à changer.

## 4. Brancher le front à l'API

### Activer les requêtes HTTP

Dans `src/app/app.config.ts`, le front ajoutera `provideHttpClient()`. Les requêtes passeront par un service qui lit l'adresse de base de l'API dans un fichier de configuration.

### Éviter les erreurs CORS en développement

Le front tourne sur `http://localhost:4200` et l'API sur une autre adresse : le navigateur bloquera les requêtes tant que l'API n'autorise pas l'origine du front. Deux solutions :

1. **Côté API** : autoriser l'origine `http://localhost:4200` (en-têtes CORS), avec les cookies si on utilise une session par cookie.
2. **Côté front** : un proxy de développement. On crée `proxy.conf.json` à la racine :

```
{
  "/api": {
    "target": "http://localhost:8080",
    "secure": false
  }
}
```

puis on lance `npx ng serve --proxy-config proxy.conf.json`. Le front appelle alors `/api/...` et le proxy redirige vers votre serveur. Adaptez le port 8080 au vôtre.

En production, mieux vaut servir le front et l'API sous le même domaine.

### Authentification

Le front ne doit jamais décider seul du rôle. À la connexion, l'API renvoie l'identité et le rôle, et vérifie le rôle à chaque requête suivante.

Option recommandée : une **session par cookie** **`HttpOnly`**, **`Secure`**, **`SameSite`**. Le front enverra ses requêtes avec les cookies activés, et ne gardera pas de jeton lisible par le JavaScript.

Si vous préférez un jeton (JWT), précisez-le : le front l'enverra dans l'en-tête `Authorization`. Un jeton gardé dans le navigateur peut être volé en cas de faille XSS, donc prévoyez une durée courte.

## 5. Valeurs fixes que le front utilise

- **Rôles** : `etudiant`, `chef`, `admin` (écrits exactement ainsi).
- **Types d'archive** : `Devoir`, `Test`, `Épreuve de session`.
- **Niveaux** : `Licence 1`, `Licence 2`, `Licence 3`.
- **Semestres** : Licence 1 = Semestre 1 et 2, Licence 2 = Semestre 3 et 4, Licence 3 = Semestre 5 et 6.
- **Années académiques** : par exemple `2024 - 2025`.

- **Filières** (5) : Génie Informatique, Réseaux et systèmes, Réseaux et télécommunications, Électricité industrielle, Électronique, électrotechnique et automatisme.

Aujourd'hui le front repère les filières et les modules par leur **nom**. Il est préférable que l'API fournisse aussi des **identifiants** : le front s'adaptera.

## 6. Formats de données attendus (JSON)

**Utilisateur connecté** (réponse de la connexion) :

```
{ "prenom": "Moussa", "nom": "Traoré", "role": "chef" }
```

**Archive** :

```
{
  "id": 1,
  "titre": "Algorithmique - Devoir 1",
  "type": "Devoir",
  "filiere": "Génie Informatique",
  "module": "Algorithmique",
  "niveau": "Licence 1",
  "semestre": "Semestre 2",
  "annee": "2024 - 2025",
  "date": "2024-10-15",
  "fichier": "/api/archives/1/fichier"
}
```

La date doit être au format ISO ( `AAAA-MM-JJ` ) : le front la mettra en forme.

**Module** :

```
{ "id": 3, "nom": "Algorithmique", "filiere": "Génie Informatique", "niveau":
"Licence 1", "semestre": "Semestre 2", "credits": 6 }
```

**Compte** (liste des utilisateurs, admin) :

```
{ "id": 2, "prenom": "Moussa", "nom": "Traoré", "identifiant": "chef.info",
"role": "chef", "actif": true }
```

**Téléchargement** (historique de l'utilisateur) :

```
{ "titre": "Algorithmique - Devoir 1", "fichier": "/api/archives/1/fichier",
"date": "2024-10-15" }
```

# 7. Routes attendues

Toutes les routes, sauf connexion et inscription, exigent une session valide.

| Méthode | Route | Rôles autorisés | Rôle de la route |
|---|---|---|---|
| POST | /auth/connexion | public | Identifiant et mot de passe, renvoie l'utilisateur |
| POST | /auth/inscription | public | Crée un compte étudiant uniquement |
| POST | /auth/deconnexion | connecté | Ferme la session |
| PUT | /auth/mot-de-passe | connecté | Ancien et nouveau mot de passe |
| GET | /filieres | connecté | Liste des filières |
| GET | /modules?filiere=&niveau= | connecté | Liste des modules, filtrable |
| POST | /modules | admin | Créer un module |
| DELETE | /modules/{id} | admin | Supprimer un module |
| GET | /archives?filiere=&module=&niveau=&annee=&type=&q= | connecté | Recherche d'archives |
| GET | /archives/{id}/fichier | connecté | Télécharge un fichier et l'ajoute à l'historique |
| POST | /archives | chef, admin | Ajouter une archive (multipart : champs et fichier) |
| GET | /programmes?filiere=&semestre= | connecté | Image du programme |
| PUT | /programmes | chef, admin | Ajouter ou remplacer une image (multipart) |
| GET | /moi/telechargements | connecté | Historique de l'utilisateur |
| DELETE | /moi/telechargements | connecté | Vider l'historique |
| GET | /utilisateurs?q=&role= | admin | Liste des comptes |
| POST | /utilisateurs | admin | Créer un compte avec un rôle |
| PATCH | /utilisateurs/{id} | admin | Changer le rôle ou activer/désactiver |

## 8. Règles de sécurité à appliquer côté serveur

Le front fait des vérifications pour le confort de l'utilisateur, mais **elles se contournent**. Le serveur doit tout refaire.

- **Injection SQL** : requêtes préparées (paramétrées) partout, aucune concaténation de texte venant de l'utilisateur.
- **Mots de passe** : hachés avec bcrypt ou argon2, jamais stockés ni renvoyés en clair.
- **HTTPS** obligatoire en production.
- **Rôles** : vérifiés à chaque requête. Le rôle n'est jamais choisi à l'inscription : tout nouveau compte est `etudiant`.
- **Chef de filière** : il ne doit agir que sur **sa** filière. Le front ne le limite pas.
- **Administrateur** : il ne doit pas pouvoir se retirer son rôle ni se désactiver.
- **Validation** : identifiant de 3 à 30 caractères (lettres, chiffres, point, tiret, souligné), mot de passe de 8 caractères minimum, longueurs maximales sur tous les champs.
- **Fichiers** : vérifier le **type réel** du fichier (pas seulement l'extension), limiter la taille (10 Mo pour les archives, 5 Mo pour les programmes), générer soi-même le nom de stockage, et stocker hors du dossier public.
- **Tentatives de connexion** : limiter leur nombre pour freiner les attaques par essais répétés.

## 9. Codes de réponse et erreurs

| Code | Quand |
|---|---|
| 200 / 201 | Succès |
| 400 | Données invalides |
| 401 | Non connecté : le front renvoie vers la connexion |
| 403 | Rôle insuffisant |
| 404 | Ressource introuvable |
| 409 | Conflit (identifiant ou module déjà existant) |
| 413 | Fichier trop lourd |

Pour les erreurs, renvoyez un message lisible par l'utilisateur, par exemple `{ "message": "Cet identifiant est déjà utilisé." }`. Le front l'affichera tel quel.

## 10. Points à décider ensemble

1. Identifiant simple ou adresse e-mail pour se connecter.
2. Session par cookie ou jeton.
3. Mot de passe initial des comptes créés par l'administrateur (mot de passe temporaire à changer à la première connexion ?).
4. Inscription ouverte à tous, ou validée par l'administrateur.
5. Comment le chef de filière est rattaché à sa filière.
6. Où et comment les fichiers sont stockés.

## 11. Contact et suite

Une fois les routes disponibles, même avec des données de test, le front remplacera le contenu de `donnees.ts` et de la connexion. Merci de communiquer l'adresse de base de l'API, les formats réels des réponses et tout écart avec ce document.